ENTERPRISE AI SUPPORT PLATFORM

Backend Engineering Reference

Enterprise AI Support Platform

Backend Architecture & Technical Design Document

VERSION 1.0 — ARCHITECTURE LOCKED

Prepared for: Enterprise Developers • Solution Architects • AI Engineers • New Team Members • Future Contributors
Scope: ai-support-platform/backend (FastAPI, Python 3.14)
Status: Permanent technical reference — architecture frozen at this revision

Table of Contents

1. Executive Summary

1.1 Purpose of the Project

The Enterprise AI Support Platform is a FastAPI backend that provides the operational core of a multi-tenant customer support product: organizations, teams, users, projects, customers, and support tickets, augmented by an artificial-intelligence layer intended to ground conversational and retrieval capabilities in an organization's own knowledge base. The system is designed as a set of independently understandable, feature-scoped modules rather than a single monolithic domain model, so that a new engineer can open one folder and understand one capability end-to-end without first internalizing the entire codebase.

1.2 Business Problems Solved

- Multi-tenant organization management — every business entity is scoped to an **Organization**, giving each customer of the platform an isolated operational space.
- Structured support-ticket workflows — tickets, comments, attachments, SLA policies, and automation workflows give support teams a full case-management lifecycle.
- Role-based access control — a permission/role model lets each organization define who can create, read, update, delete, or assign records, independent of hardcoded logic.
- Auditable operations — every sensitive action can be recorded in an immutable audit log for compliance and incident investigation.
- AI-assisted support — a dedicated `app/ai` layer is architected to let support agents and customers get answers grounded in organizational knowledge rather than a model's raw, unverified knowledge.

1.3 Enterprise Objectives

- **Separation of concerns** — each business capability owns its own data access, business rules, and HTTP surface, so changes in one module cannot silently break another.
- **Predictable extension** — new business modules and new AI capabilities follow the same file-per-responsibility shape, so the cost of adding a seventeenth module is the same as the cost of adding the second.
- **Operational transparency** — structured logging, a global exception-handling contract, and a consistent response envelope make failures diagnosable in production.
- **Testability by construction** — dependency injection at every layer means every service and repository can be exercised in isolation, and every router can be exercised against a real (test) database via dependency overrides.

1.4 AI Capabilities

The `app/ai` package is architected around the standard Retrieval-Augmented Generation (RAG) pipeline: documents are registered, an ingestion job tracks their processing into chunks, chunks are turned into vector embeddings, embeddings are queried at retrieval time, and a RAG service is meant to combine retrieved context with a prompt sent to a large-language-model provider, with the resulting conversation persisted for audit and continuity. A dedicated multi-provider gateway (`app/ai/providers`) abstracts OpenAI, Anthropic, Gemini, Groq, Azure OpenAI, and Ollama behind one interface, plus a deterministic mock provider for testing. Section 4 documents each AI module's current implementation status in detail, including which parts of this pipeline are fully

wired today and which are architected but not yet functionally complete — this document is a technical reference, and an inaccurate account of what is actually implemented would be a disservice to the engineers who rely on it.

1.5 Why This Architecture Was Chosen

The codebase follows **Clean Architecture** principles adapted into a pragmatic, FastAPI-idiomatic shape: vertical feature slices (one folder per business capability) instead of horizontal technical layers spread across the whole codebase (no single giant `models.py` or `services.py`). Within each slice, the classic four-layer flow — Router → Service → Repository → Database — is enforced consistently: routers own HTTP concerns only, services own business rules and raise typed domain exceptions, repositories own persistence and nothing else. This was chosen over a single shared service/repository layer because it keeps the blast radius of any change contained to one folder, lets different modules evolve at different speeds, and makes onboarding tractable — a new engineer assigned to the “tickets” feature never needs to read the AI or workflow code to be productive.

Figure 1.1 — High-Level Architecture Overview

Figure 1.1 — Every business and AI module sits on the same shared infrastructure and follows the same Router → Service → Repository → Database flow.

2. Technology Stack

The platform is built on a modern, typed Python stack. The table below lists every core technology and pattern used, with the specific role it plays in this codebase.

Technology	Role in This Codebase
Python 3.14	Language runtime. The codebase declares `requires-python >= 3.14` and uses modern syntax throughout: PEP 604 union types (`str | None`), PEP 695 generic classes (e.g. `SuccessResponse[T]` in app/common/responses.py), and `from __future__ import annotations` in every module.
FastAPI	Web framework. Every business and AI capability exposes an `APIRouter` mounted onto a single composition root (`app/api/v1/router.py`). FastAPI's dependency-injection system (`Depends()`) is the backbone of the Repository/Service wiring described in Section 11.
SQLAlchemy 2.x	ORM. All models use the SQLAlchemy 2.0 declarative style with typed `Mapped[...]` columns, built on a single shared `DeclarativeBase` (`app/database/base.py`) with an Alembic-safe naming convention for every constraint type.
Alembic	Migration framework. The environment is fully wired (`app/database/migrations/env.py` reads `Base.metadata` and `settings.DATABASE_URL`, supports offline and online modes), but as of this revision no `versions/` directory or `alembic.ini` exists in the repository — schema is currently produced via `Base.metadata.create_all()`, not a committed migration history. See Section 9 and Section 19.
PostgreSQL	Primary datastore (production). The default connection string in `app/config/settings.py` targets `postgresql+psycopg://`. The test suite substitutes a local SQLite file via dependency override (Section 15) so the full suite can run without a live Postgres instance.
Pydantic v2	Schema validation and serialization for every request/response model, plus application configuration via `pydantic-settings`' `BaseSettings` (`app/config/settings.py`).
JWT	Stateless authentication. Bearer tokens are issued at login and verified on every protected request via `PyJWT`. Section 8 documents an important inconsistency between two parallel JWT implementations found in the codebase.
RBAC	Role-Based Access Control. A four-table model (Permission, Role, RolePermission, UserRole) plus a `require_permission(resource, action)` dependency factory (`app/rbac`) enforces fine-grained authorization, currently adopted by 4 of the 21 mounted routers — see Section 5 and Section 8.
Dependency Injection	FastAPI's `Depends()` composes every request's Repository → Service chain from a per-request database session, with each module's own `dependencies.py` defining named `Annotated[...]` aliases. See Section 11.
Repository Pattern	Every module isolates SQLAlchemy queries behind a Repository class with no business logic — services never issue raw queries themselves. See Section 12.
Service Layer	Every module's business rules, validation, and domain-exception raising live in a Service class that depends only on repositories, never on the HTTP layer. See Section 13.
Clean Architecture	Enforced dependency direction — Router → Service → Repository → Database, one-way, with no reverse imports (no repository imports a service; no service imports a router).

Technology	Role in This Codebase
SOLID Principles	Single Responsibility (one file per concern per module), Open/Closed (the `AIProvider` and `BaseStorageProvider`/`BaseEmailProvider` abstractions let new providers be added without modifying callers), Liskov Substitution (all provider implementations honor the same abstract base contract), Interface Segregation (narrow, per-module dependency aliases rather than one giant service locator), Dependency Inversion (services depend on repository abstractions injected at the router boundary, not concrete instances they construct themselves).
Pytest	Testing framework. 622+ test functions across 90+ test files, organized by module and by layer (repository / service / router). See Section 15.
Ruff	Linting. Configured with the `E, F, I, UP, B, C4, SIM, N, ANN, D` rule families and Google-convention docstring checks; the full codebase passes with zero issues as of this revision.
Black	Code formatting, 88-column line length, Python 3.14 target; the full codebase is fully formatted with zero pending changes as of this revision.
MyPy	Static type checking in `strict` mode (disallows untyped/incomplete defs, warns on `Any` returns and unused ignores); the full codebase passes with zero issues across 427 source files as of this revision.
Docker Ready	A `docker-compose.yml` defines `backend`, `postgres` (17), and `redis` (8) services. As of this revision the referenced `Dockerfile` does not yet exist in the repository — see Section 19 for the deployment gap this creates.

3. Complete Folder Architecture

The backend package root is `ai-support-platform/backend/app/`. Every folder at this level is either a piece of shared infrastructure, a business module, the AI package, or the test suite. There is no generic “modules” catch-all directory — every capability is a first-class, top-level package.

```
app/
|-- main.py                  FastAPI app instance, lifespan wiring, root route
|
|-- api/                     Composition root
|   `-- v1/router.py         Mounts all 21 feature routers under /api/v1
|
|-- core/                    Framework-level shared plumbing
|   |-- dependencies.py      SettingsDependency, DatabaseDependency aliases
|   |-- exceptions.py        AppException hierarchy (7 typed exceptions)
|   `-- lifespan.py          startup()/shutdown() task registration
|
|-- common/                  Cross-cutting FastAPI concerns
|   |-- exceptions.py        register_exception_handlers()
|   |-- pagination.py        PaginationParams, get_pagination()
|   `-- responses.py         SuccessResponse / ErrorResponse / PaginatedResponse
|
|-- config/                  Application configuration
|   |-- settings.py          Settings (BaseSettings) - env-driven config
|   |-- logging.py           configure_logging(), get_logger()
|   |-- security.py          Parallel JWT/password module (see Section 8)
|   `-- constants.py         API prefix, pagination defaults, role names
|
|-- database/                Persistence plumbing
|   |-- base.py              DeclarativeBase + naming convention
|   |-- engine.py            SQLAlchemy engine (pool_size=10, max_overflow=20)
|   |-- session.py           get_db() request-scoped session dependency
|   |-- types.py             GUID - cross-dialect UUID column type
|   `-- migrations/          Alembic scaffold (env.py; no versions/ committed)
|
|-- models/                  Centrally-owned shared entities
|   |-- base.py, mixins.py   BaseModel, TimestampMixin, OrganizationMixin
|   |-- organization.py, user.py, team.py, project.py, ticket.py
|   `-- permission.py, role.py, role_permission.py, user_role.py
|
|-- repositories/            Generic + shared-entity repositories
|   |-- base.py              Generic BaseRepository[ModelType]
|   `-- organization.py, project.py, user.py, permission.py, role.py, ...
|
|-- ai/                      AI capability package (Section 4)
|   |-- constants.py, schemas.py, exceptions.py   shared AI-wide vocabulary
|   |-- chat/  documents/  embeddings/  ingestion/  knowledge/
|   `-- prompts/  providers/  rag/  retrieval/  vectorstore/
|
|-- <17 business modules>    Section 5 - one vertical slice per folder
|   auth/  organizations/  teams/  users/  projects/  customers/
|   tickets/  comments/  attachments/  notifications/  audit/
|   analytics/  sla/  rbac/  files/  email/  workflows/
|
|-- workers/                 Celery task scaffolding (5 stub files - Section 19)
|
`-- tests/                   Single test root (pytest testpaths = app/tests)
    |-- conftest.py          ~1070 lines of shared fixtures
    |-- database.py          SQLite test-database setup
    `-- <mirrors the tree above, one folder per module>
```

Figure 3.1 — Top-level layout of app/. Every top-level folder is either shared infrastructure, a business module, the AI package, or the test suite.

3.1 Purpose of Every Folder

Folder	Purpose
app/api/	Single composition root. The only file that knows about every other module's router — this is intentional: it is the one place a new module must be registered to become reachable.
app/core/	Framework-level primitives with zero business meaning: the shared exception base class, shared DB/Settings dependency aliases, and application lifespan hooks.
app/common/	Cross-cutting concerns that are about HTTP/response shape, not business rules: global exception handler registration, pagination parameters, and standard response envelopes.
app/config/	Everything that varies by deployment environment: connection strings, secrets, logging setup, and fixed application-wide constants.
app/database/	Everything about how a SQLAlchemy session is created and how the schema is defined at the ORM-metadata level, independent of any one module's tables.
app/models/ + app/repositories/	The five entities used across many modules at once (Organization, User, Team, Project, Ticket) plus the RBAC tables are centralized here rather than owned by a single feature folder, because no single module is their sole owner.
app/ai/	Every AI-specific capability, organized the same way as a business module (one folder per capability) but grouped under a common namespace because they share AI-specific vocabulary (provider enums, prompt/response schemas) that no business module needs.
17 business module folders	Each is a fully self-contained vertical slice: its own models (where it owns a table), schemas, repository, service, router, dependencies, exceptions, and constants.
app/workers/	Reserved location for asynchronous/background task definitions (Celery). Currently scaffolding only — see Section 19.
app/tests/	Mirrors the source tree one-for-one; this is the only test root recognized by the project's pytest configuration.

4. AI Architecture

The AI package (`app/ai/`) is organized as ten vertical slices sharing three root-level files: `constants.py` (provider/model/prompt-role enums), `schemas.py` (provider-agnostic request/response contracts), and `exceptions.py` (the AI error hierarchy). This section documents each module's purpose, data shape, API surface, and — critically for a technical reference — its actual current implementation status, verified by reading the code rather than assumed from the folder name.

> **■ Implementation Status Notice**
>
> Several AI modules described below are architecturally complete (typed models, schemas, CRUD services, REST endpoints) but functionally incomplete: similarity search is not yet implemented (scores are hardcoded), no embedding-model provider is called to generate real vectors, and the RAG generation endpoint returns a placeholder string rather than an LLM-generated answer. This is stated explicitly per module below and summarized in the pipeline trace at the end of this section, so that engineers extending this system know exactly what remains to reach full functionality.

4.1 Chat

chat prefix: /ai/chat (built, not mounted) | table: ai_conversations, ai_conversation_messages

Purpose: Conversation and message persistence with an end-to-end send-message-and-get-a-real-LLM-reply flow. The most functionally complete AI module in the codebase.

Inputs: ChatRequest (conversation_id, message text) plus conversation metadata at creation (organization_id, customer_id, ticket_id, provider, model).

Outputs: ChatResponse containing the created conversation, the persisted user message, and the persisted assistant message (with token_count and latency_ms).

Business value: This is the actual agent/customer-facing conversational surface — persisted, auditable, and tied to a ticket/customer for full support-case context.

Dependencies: Calls app.ai.providers.registry.get_provider(...) directly to invoke an LLM. Uses its own chat/prompts/ builder (distinct from the root-level app/ai/prompts) to construct the message list, prepending a static system prompt.

Interaction with other AI modules: Real, working dependency on providers only. Does NOT call rag, retrieval, vectorstore, embeddings, or knowledge — replies are generated from conversation history plus a static system prompt, with no knowledge-base grounding.

Status: IMPLEMENTED, but its router is not included in app/api/v1/router.py or app/main.py — the fully-working chat API is not reachable from the running application today; it is exercised only by its own test suite.

4.2 Documents

documents prefix: /api/v1/ai/documents | table: ai_documents

Purpose: Metadata registry for uploaded source files destined for a knowledge base — filename, storage location, checksum, version, and a status/chunk_count/embedding_count lifecycle.

Inputs: DocumentCreateRequest (filename, storage_path, content_type, checksum, knowledge_id).

Outputs: DocumentResponse; DocumentStatisticsResponse (counts by status).

Business value: Auditable system-of-record for what source material has entered a knowledge base; checksums support de-duplication and change detection.

Dependencies: Self-contained; has a foreign key to knowledge_bases but does not import the knowledge module's code.

Interaction with other AI modules: None at the code level in either direction — not called by ingestion, and does not call embeddings itself.

Status: CRUD fully implemented. Does not perform any file I/O or parsing — expects storage_path to already point at a stored file.

4.3 Ingestion

ingestion prefix: /api/v1/ai/ingestion | table: ai_ingestion_jobs

Purpose: Job-tracking/state-machine for the document processing pipeline (registered -> processing -> completed/failed), with counters for chunks and embeddings created.

Inputs: IngestionCreateRequest (document_id, chunk_size, chunk_overlap, file_type).

Outputs: IngestionResponse; job statistics (total/completed/processing/failed).

Business value: Gives operators visibility into ingestion pipeline health and throughput — essential for diagnosing why content isn't reaching the AI answer surface.

Dependencies: Self-contained; references document_id but does not import the documents module's code.

Interaction with other AI modules: None. No code path calls embeddings to actually generate embeddings, despite tracking an embeddings_created counter.

Status: State-machine/CRUD implemented (start_job/complete_job/fail_job). No chunking algorithm exists anywhere in app/ai/ — chunk counters are updated by the caller, not computed from real document content.

4.4 Embeddings

embeddings prefix: /api/v1/embeddings | table: embeddings

Purpose: Central vector data store — a polymorphic table (source_type/source_id) that can back semantic search over knowledge articles, tickets, comments, or conversations alike.

Inputs: EmbeddingCreate (organization_id, source_type, source_id, content, provider, model).

Outputs: EmbeddingResponse; list/get/update/delete, all organization-scoped.

Business value: Provides the single shared vector store that every retrieval-adjacent module reads from, avoiding duplicated storage per feature.

Dependencies: Self-contained; deduplicates on (organization_id, source_id).

Interaction with other AI modules: The most depended-upon AI module: vectorstore, retrieval, and rag all import app.ai.embeddings.models.Embedding directly to query the table — bypassing this module's own service/repository layer entirely.

Status: CRUD fully implemented, but create() never calls an embedding-model provider — it persists vector=[] and status=PENDING. Real vector generation is not implemented.

4.5 Knowledge

knowledge prefix: /api/v1/knowledge | table: knowledge_bases

Purpose: Owns the AI-facing knowledge-base container concept — a named, access-scoped collection that documents, embeddings, and RAG generations attach to via knowledge_id.

Inputs: KnowledgeCreate (name, description, status, visibility: private/organization).

Outputs: KnowledgeResponse; list/get/update/delete, organization-scoped, name-unique per organization.

Business value: Provides the organizational/access-control boundary for grouping support content into distinct knowledge domains (e.g. per-product or per-team), a baseline enterprise governance requirement.

Dependencies: Self-contained CRUD; no cross-module imports.

Interaction with other AI modules: Referenced by documents, embeddings, and rag via foreign key only — none of those modules import this module's Python code.

Status: Fully implemented CRUD. This module replaced an earlier, separate business 'knowledge base articles' module that shared its URL prefix — see Section 21.

4.6 Prompts

prompts prefix: n/a (library, no router) | table: none

Purpose: A static prompt-template utility (PromptLibrary) offering canned system prompts: system(), support_assistant(), summarizer(), classifier(), translator().

Inputs: None — static factory methods.

Outputs: PromptMessage objects using the shared app.ai.schemas.PromptMessage/constants.PromptRole types.

Business value: Would give a single, reusable, versionable location for system prompts, avoiding prompt text scattered across services.

Dependencies: Reads app.ai.constants.PromptRole and app.ai.schemas.PromptMessage; no database table, no HTTP surface.

Interaction with other AI modules: Not imported by chat, rag, or providers today — chat has its own separate chat/prompts/ package (builder.py, system.py, templates.py) that duplicates this concept independently.

Status: Implemented but effectively unused — dead code from the perspective of the rest of the AI pipeline.

4.7 Providers

providers prefix: /api/v1/ai | table: none (static configuration)

Purpose: Two responsibilities in one package: (1) the LLM provider abstraction layer — an AIProvider ABC, a registry factory, and one concrete class per vendor; (2) a generic, provider-agnostic HTTP gateway (chat/completion/health/configuration/streaming).

Inputs: AIRequest (provider, model, request_type, messages, temperature, max_tokens, stream).

Outputs: AIResponse (content, token usage, finish_reason) or a text/event-stream for the streaming endpoint.

Business value: Centralizes multi-vendor LLM access behind one interface, enabling vendor fail-over, cost control, and deterministic testing via the mock provider.

Dependencies: Self-contained; owns no database table (configuration/health responses are static).

Interaction with other AI modules: The most depended-upon module for LLM access: chat imports app.ai.providers.registry.get_provider directly, in-process, bypassing this module's own HTTP gateway entirely.

Status: Only OpenAI and the Mock provider have functional generate()/stream() implementations. Anthropic, Gemini, Groq, Ollama, and Azure OpenAI providers are registered in the factory but raise NotImplementedError for both methods.

4.8 RAG (Retrieval-Augmented Generation)

rag `prefix: /api/v1/ai/rag | table: rag_generations (defined, unused)`

Purpose: Intended orchestrator: accept a query, retrieve relevant context, and call an LLM to produce a grounded answer with citations — the core value proposition of an AI support platform.

Inputs: RAGRequest (query, provider, top_k, temperature, max_tokens).

Outputs: RAGResponse (provider, model, answer, citations, token usage).

Business value (as designed): Would let agents/customers receive answers sourced from the organization's own knowledge base rather than a model's unverified parametric knowledge — directly reduces hallucination risk in a support context.

Dependencies: Imports app.ai.embeddings.models.Embedding directly for its own repository queries.

Interaction with other AI modules: Does NOT call retrieval, vectorstore, providers, or prompts — the module intended to be the pipeline's glue currently connects to none of the modules it should orchestrate.

Status: generate() returns the literal placeholder string "This is a placeholder AI-generated response." with all-zero token usage; citations are built from whichever embedding rows happen to be paged in, with score hardcoded to 1.0. No LLM call is made. The rag_generations table is defined but no code path ever writes to it.

4.9 Retrieval

retrieval `prefix: /api/v1/ai/retrieval | table: retrieval_sessions (defined, unused)`

Purpose: Query-time document-fetch step intended to ground LLM answers: semantic, hybrid, and metadata-filtered retrieval strategies.

Inputs: RetrievalRequest / HybridRetrievalRequest / MetadataSearchRequest (query text, top_k, and for hybrid: keyword/semantic weights).

Outputs: RetrievalResponse — a list of RetrievedDocumentResponse(id, content, score, metadata).

Business value (as designed): Would provide the pluggable retrieval strategy layer that determines what context reaches the LLM — central to answer quality and hallucination reduction.

Dependencies: Imports app.ai.embeddings.models.Embedding directly, same pattern as vectorstore.

Interaction with other AI modules: Not called by rag despite the naming implying it should be. The retrieval_sessions table is defined but never written to.

Status: score is hardcoded to 1.0 for every result; hybrid_retrieve accepts keyword_weight/semantic_weight parameters that are never used in the query logic. Functionally returns 'the first top_k embedding rows,' not ranked results.

4.10 Vector Store

vectorstore **prefix: /api/v1/vectorstore | table: none (reads embeddings table)**

Purpose: A search facade over the embeddings table with a pluggable VectorStoreProvider Protocol, intended to allow swapping the underlying vector infrastructure (pgvector, Pinecone, Qdrant, Weaviate, Chroma, Milvus, Elasticsearch are all named in constants.py) without changing calling code.

Inputs: SemanticSearchRequest / SimilarEmbeddingsRequest / HybridSearchRequest.

Outputs: VectorSearchResponse; provider list; vector-store statistics.

Business value (as designed): Would let the platform change vector-database infrastructure to match an enterprise customer's existing investment without touching application code.

Dependencies: Imports app.ai.embeddings.models.Embedding directly.

Interaction with other AI modules: None — not called by retrieval or rag despite functional overlap with both.

Status: similarity_score is hardcoded to 1.0 for every result; the providers endpoint only ever returns a static 'openai' entry; statistics' source/knowledge counts are hardcoded to 0. No real vector comparison is performed.

4.11 AI Pipeline Trace — Upload to Chat Response

The following traces the intended document-to-answer pipeline against what the code actually does at each step, citing the real service methods involved. This is the single most load-bearing fact for any engineer planning to extend the AI layer.

```
STEP                    INTENDED BEHAVIOR              ACTUAL BEHAVIOR (as coded)
-------------------------------------------------------------------------------
1. Upload               File bytes stored               Outside app/ai/ entirely -
                                                          handled by files/attachments

2. Document registration DocumentService                 Persists metadata row.
                          .create_document()              Does NOT call ingestion.

3. Ingestion tracking    IngestionService                State machine only.
                          .create_job()/.start_job()      No chunking algorithm exists
                                                          anywhere in app/ai/.

4. Embedding generation  AIEmbeddingService.create()     Persists vector=[],
                                                          status=PENDING.
                                                          No embedding-model API
                                                          call is made.

5. Vector storage        Dedicated vector index          IS the embeddings table -
                                                          vectorstore/retrieval/rag
                                                          all read it directly.

6. Retrieval             Ranked semantic search          score hardcoded to 1.0.
                          (retrieval / vectorstore)       Returns "first N rows,"
                                                          not ranked results.

7. RAG generation        Retrieve + prompt + LLM call    Returns hardcoded
                          (rag.generate)                  placeholder text.
                                                          No LLM is called here.

8. Chat response         Grounded, cited LLM answer      REAL LLM call via
                          (chat.send_message)             providers.registry.
                                                          Does NOT call rag or
                                                          retrieval - grounded
                                                          only in conversation
                                                          history + static prompt.
                                                          Router not mounted -
                                                          unreachable via the
                                                          live API today.
-------------------------------------------------------------------------------
NET: The data model and API surface for the full pipeline exist and are well-typed.
The cross-module glue that would make it function end-to-end - real chunking, real
embedding generation, real vector similarity, and RAG-grounded chat - is the
remaining engineering work. See Section 20 (Future Roadmap).
```

Figure 4.1 — Verified pipeline trace, current revision. Each row cites the actual method call chain found in source.

5. Business Modules

All 17 business modules are mounted under `/api/v1` via `app/api/v1/router.py`. Five entities (Organization, User, Team, Project, Ticket) plus the four RBAC tables are centrally modeled in `app/models/` because they are referenced by many modules at once; the remaining eleven modules own their tables directly.

5.1 Organizations

organizations prefix: /organizations (superuser-only) | table: organizations (central model)

Responsibilities: Root tenant entity CRUD — every other business record is scoped to an organization.

Database: id, name (unique), code (unique), email, phone, website, address fields, timezone, is_active, soft-delete via is_deleted/deleted_at.

Endpoints: POST /, GET /, GET /{id}, PATCH /{id}, DELETE /{id} — all guarded by CurrentSuperuserDependency.

Business logic: Enforces uniqueness on both name and code at create and update time; delete is soft.

Relationships: Parent of tickets, users, sla_policies (cascade delete-orphan on all three).

Dependencies / known issue: Two independent, unreconciled OrganizationRepository implementations exist for this entity — see Section 9.

5.2 Teams

teams prefix: /teams | table: teams (central model)

Responsibilities: Team CRUD within an organization, with an optional lead user.

Database: id, organization_id (FK), lead_id (FK->users, nullable), name, code (unique), description, is_active.

Endpoints: POST /, GET /{id}, GET /organization/{organization_id}, PATCH /{id}, DELETE /{id}.

Business logic: Validates the parent organization exists; enforces name uniqueness per-organization and code uniqueness globally; validates lead_id resolves to a real user.

Relationships: Belongs to Organization; optional lead User.

Dependencies: app.repositories.organization.OrganizationRepository (the BaseRepository-based one), app.repositories.user.UserRepository.

5.3 Users

users prefix: /users (RBAC: user:*) | table: users (central model)

Responsibilities: User account CRUD, password hashing at creation.

Database: id, organization_id (FK, RESTRICT), email (unique), username (unique), full_name, password_hash, is_active, is_superuser.

Endpoints: POST /, GET / (paginated), GET /{id}, PATCH /{id}, DELETE /{id} — guarded via require_permission("user", <action>).

Business logic: Hashes password on create; enforces email/username uniqueness; blocks mutation of id/organization_id on update via an immutable_fields set.

Relationships: Belongs to Organization; referenced by Ticket (created_by/assigned_to), Team (lead_id), Project (owner_id), UserRole, Comment, Attachment, File, Email, Notification.

Dependencies: app.auth.password, app.organizations.repository.OrganizationRepository (the hand-written one).

5.4 Projects

projects prefix: /projects (no auth guard) | table: projects (central model)

Responsibilities: Project CRUD within an organization.

Database: id, organization_id, name (unique), key (unique, <=10 chars), description, owner_id (FK->users, RESTRICT), is_active.

Endpoints: POST /, GET / (paginated), GET /{id}, PUT /{id}, DELETE /{id}.

Business logic: Enforces name and key uniqueness at creation; update does not re-validate uniqueness on rename.

Relationships: Belongs to Organization; owner_id references User.

Dependencies / known issue: No authentication or permission dependency is present on this router's endpoints, unlike most other modules.

5.5 Customers

customers prefix: /customers | table: customers (module-owned)

Responsibilities: External customer/contact records associated with an organization.

Database: id, organization_id (CASCADE), name, company_name, email (unique), phone, address fields, customer_type (individual/business), status (active/inactive/suspended).

Endpoints: POST /, GET / (paginated), GET /{id}, PUT /{id}, DELETE /{id} — guarded by CurrentActiveUserDependency.

Business logic: Enforces email uniqueness at create and on update (only when the email is changing).

Relationships: Belongs to Organization. No direct FK from Ticket to Customer exists in the current model.

Dependencies: Self-contained.

5.6 Tickets

tickets prefix: /tickets (RBAC: ticket:*) | table: tickets (central model)

Responsibilities: The core support case entity — status/priority-tracked work items.

Database: id, organization_id, created_by/assigned_to (FK->users), title, description, status (default open), priority (default medium), is_active.

Endpoints: Only GET / and POST / are wired, despite the service implementing full CRUD (get/update/delete exist in TicketService but have no corresponding routes).

Business logic: Enforces title uniqueness globally (not per-organization); status values: open/in_progress/pending/resolved/closed; priority values: low/medium/high/critical.

Relationships: Belongs to Organization; creator/assignee reference User; has-one SLAEvent; Comments and Attachments FK to ticket_id.

Dependencies: app.rbac.dependencies.require_permission.

5.7 Comments

comments prefix: /comments | table: comments (module-owned)

Responsibilities: Threaded commentary on tickets, with public/internal visibility.

Database: id, ticket_id (CASCADE), organization_id (CASCADE), author_id (FK->users, CASCADE), content, visibility (public/internal), is_internal, is_edited, is_deleted.

Endpoints: POST /tickets/{ticket_id}, GET / (optional ticket_id filter), GET /{id}, PUT /{id}, DELETE /{id} — guarded by CurrentActiveUserDependency.

Business logic: is_internal is derived from visibility; update marks the comment as edited whenever any field actually changes.

Relationships: Belongs to Ticket, Organization, and author User.

Dependencies: Self-contained.

5.8 Attachments

attachments prefix: /attachments | table: attachments (module-owned)

Responsibilities: File attachments linked to a ticket and/or a comment.

Database: id, organization_id, ticket_id (nullable), comment_id (nullable), uploaded_by_id, filename, content_type, file_size, storage_provider, storage_key (unique), checksum (unique).

Endpoints: POST /tickets/{id}, POST /comments/{id}, GET /, GET /{id}, GET /tickets/{id}, GET /comments/{id}, PUT /{id}, DELETE /{id} — guarded by CurrentActiveUserDependency.

Business logic: Deduplicates by checksum. Defined MIME-type and 25MB size-limit constants exist but are not enforced anywhere in the service.

Relationships: Belongs to Organization; optionally to Ticket and/or Comment; uploaded_by_id references User.

Dependencies: Self-contained.

5.9 Notifications

notifications prefix: /notifications (RBAC: notification:*) | table: notifications (module-owned)

Responsibilities: In-app notification delivery and read-state tracking for the current user.

Database: id, organization_id, recipient_id (FK->users), title, message, notification_type (system/info/success/warning/error), priority, channel (in_app/email/sms/push), is_read.

Endpoints: GET /, POST /, PUT /{id}, GET /unread, GET /{id}, PATCH /{id}/read, PATCH /{id}/unread, DELETE /{id} — guarded via require_permission("notification", <action>).

Business logic: mark_as_read/mark_as_unread toggle is_read; listing is always scoped to the authenticated user.

Relationships: Belongs to Organization; recipient_id references User.

Dependencies: app.rbac.dependencies.require_permission.

5.10 Audit

audit prefix: /audit (no auth guard) | table: audit_logs (module-owned)

Responsibilities: Immutable audit trail of sensitive actions across the platform.

Database: id, organization_id, user_id (unconstrained UUID), action, entity_type, entity_id (unconstrained UUID), old_values/new_values (JSON), ip_address, user_agent, status. No updated_at — audit rows are append-only.

Endpoints: POST /, GET /, GET /search, GET /entity/{type}/{id}, GET /user/{user_id}, GET /{id}.

Business logic: search()/count() build dynamic filters from a search schema; entity/user/organization history are thin wrappers around search().

Relationships: Belongs to Organization; user_id/entity_id are intentionally unconstrained (no FK) to stay loosely coupled to whatever triggered the action.

Dependencies / known issue: No authentication guard is present on any audit route.

5.11 Analytics

analytics prefix: /analytics (no auth guard) | table: none — read-only aggregation layer

Responsibilities: Cross-module dashboard and metrics aggregation. Explicitly defines no ORM models of its own.

Database: None owned. Reads Organization, User, Project, Ticket, Workflow, Email, AuditLog, SLAPolicy, SLAEvent.

Endpoints: GET /dashboard, GET /metrics/tickets, /metrics/users, /metrics/organizations, /metrics/workflows, /metrics/sla, GET /health.

Business logic: Composes counts from raw func.count() queries into typed response schemas; SLA compliance = ((policies - breaches) / policies) * 100, clamped to >=0, defaulting to 100.0 at zero policies.

Relationships: The heaviest cross-module reader in the codebase — no writes, no state transitions.

Dependencies / known issue: Ships three unused parallel 'manager' classes (dashboard.py, metrics.py, reports.py) that duplicate AnalyticsService's logic but are never called from the router.

5.12 SLA

sla prefix: /sla (no auth guard) | table: sla_policies, sla_events (module-owned)

Responsibilities: Service-level-agreement policy definitions and per-ticket breach tracking.

Database: sla_policies: name, priority, first_response_minutes, resolution_minutes, business_hours_only, is_active. sla_events: ticket_id (unique, 1:1), policy_id, due timestamps, breach flags.

Endpoints: Policy CRUD (GET/POST/PATCH/DELETE /policies...), GET /breached, GET /tickets/{ticket_id}.

Business logic: assign_policy computes due dates from policy minute-offsets and refuses inactive policies; record_first_response/resolve_ticket timestamp events and compute breach flags — but these three methods are not exposed via any route, only policy CRUD and read paths are.

Relationships: SLAPolicy belongs to Organization, has many SLAEvents. SLAEvent belongs to a Ticket (1:1) and a Policy.

Dependencies: Consumed heavily by app.analytics.repository for breach/compliance counts.

5.13 RBAC

rbac **prefix: no REST surface — library only | table: permissions, roles, role_permissions, user_roles**

Responsibilities: Authorization: assigning roles to users, granting permissions to roles, and checking whether a user holds a given (resource, action) permission.

Database: permissions (resource+action unique), roles (organization-scoped, unique name), role_permissions and user_roles (both unique-constrained join tables).

Endpoints: None — RBAC has no router.py or schemas.py. It is consumed as a dependency by other modules' routers, not called directly by clients.

Business logic: assign_role/grant_permission validate existence and reject duplicates; assign_role additionally rejects cross-organization assignment (user.organization_id must match role.organization_id); has_permission(user_id, resource, action) checks whether any of the user's roles grants that permission.

Relationships: User -> UserRole -> Role -> RolePermission -> Permission.

Dependencies / adoption: require_permission() is used as a router guard in only 4 of 21 mounted routers (tickets, notifications, users, and rbac's own internals) — most business routers rely only on CurrentActiveUserDependency or no guard at all. See Section 8.

5.14 Files

files **prefix: /files (create requires auth; others open) | table: files (module-owned)**

Responsibilities: General-purpose file storage registry with a pluggable storage-provider abstraction.

Database: id, organization_id, uploaded_by_id (SET NULL), filename, content_type, size, checksum, storage_path, provider (local/s3/azure_blob/gcs), category, status (pending/uploading/active/failed/deleted).

Endpoints: POST /, GET /, GET /search, GET /{id}, PATCH /{id}, DELETE /{id}.

Business logic: Deduplicates by checksum; delegates physical storage to a BaseStorageProvider (currently only a LocalStorageProvider stub that does not write bytes anywhere); force-sets status=ACTIVE on creation rather than following the modeled pending/uploading states.

Relationships: Belongs to Organization; uploaded_by_id references User.

Dependencies / known issue: validators.py (size/content-type/filename checks) and storage.py helpers exist but are not called anywhere in the service.

5.15 Email

email prefix: /emails (create requires auth; others open) | table: emails (module-owned)

Responsibilities: Outbound email lifecycle tracking with a pluggable provider abstraction.

Database: id, organization_id, sender_id (SET NULL), recipient, subject, body, cc/bcc, template, provider (smtp/sendgrid/ses/mailgun/azure), status (pending/queued/sending/sent/delivered/failed/cancelled), retry_count.

Endpoints: POST /, GET /, GET /search, GET /{id}, PATCH /{id}, POST /{id}/send, POST /{id}/retry, POST /{id}/cancel, DELETE /{id}.

Business logic: send() blocks re-sending an already-sent/cancelled email and marks failures with an incremented retry_count; retry() resets to PENDING and re-sends; cancel() blocks cancelling an already-sent email.

Relationships: Belongs to Organization; sender_id references User. recipient/cc/bcc are free-text addresses, not linked to Customer or User records.

Dependencies / known issue: Delivery is delegated to a BaseEmailProvider — currently only an SMTPEmailProvider stub that always reports success without calling smtplib. A full EmailTemplates class exists but is never invoked by the service.

5.16 Workflows

workflows prefix: /workflows (no auth guard) | table: workflows, workflow_conditions, workflow_actions

Responsibilities: Configurable trigger/condition/action automation rules.

Database: workflows: name, trigger, is_active. workflow_conditions: field, operator, value. workflow_actions: action, value, execution_order.

Endpoints: POST /, GET /, GET /{id}, PATCH /{id}, DELETE /{id}, POST /{id}/activate, POST /{id}/deactivate, POST /{id}/execute.

Business logic: create_workflow inserts the parent row then each nested condition/action; update special-cases a status field mapped to is_active; execute_workflow refuses to run a disabled workflow.

Relationships: Workflow has many WorkflowConditions and WorkflowActions (cascade delete).

Dependencies / known issue: execute_workflow returns a summary of actions that would run but does not actually mutate a ticket or dispatch configured actions (e.g. ASSIGN_USER, SEND_EMAIL) — execution is simulated, not real. No other module (tickets, comments, sla) calls into workflows to fire triggers.

■ Cross-Cutting Observations Worth Tracking

Two independent, live OrganizationRepository implementations exist (Section 9). Auth-guard adoption is inconsistent across routers — some use RBAC's require_permission, some use a bare active-user check, and several (projects, audit, analytics, sla, workflows, most of files/email) have no auth dependency at all on their router signatures. Several modules ship validated-but-unused helper code (analytics' three manager classes, files' validators.py, email's EmailTemplates) — real implementations that were never wired to their callers.

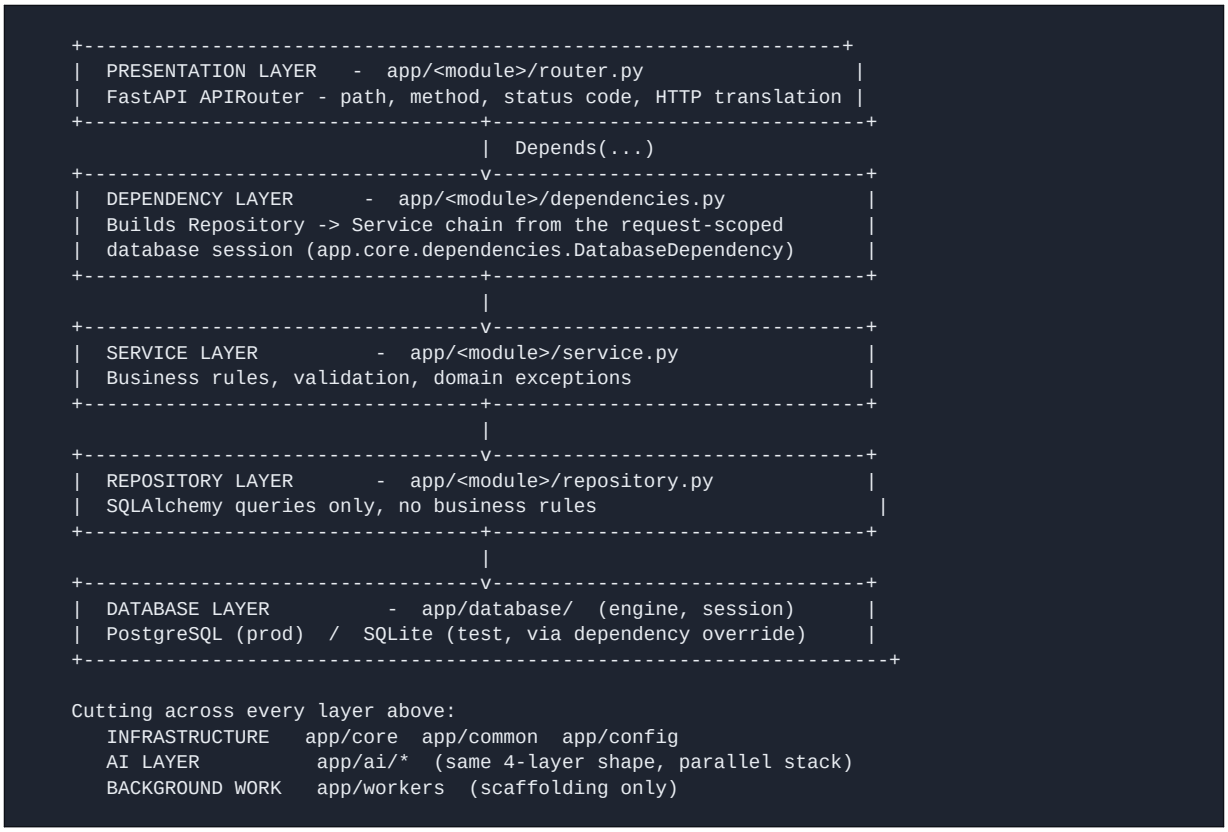
6. Layered Architecture

Every module in this codebase follows the same eight-layer flow. No exception to this direction was found in any mounted module: routers never call repositories directly, services never import a router, and repositories never import a service.

6.1 The Layers

Layer	Responsibility
Presentation / API Routers	FastAPI APIRouter objects. HTTP concerns only: path/method declaration, request parsing, status codes, and translating domain exceptions to HTTP responses. No business logic.
Dependency Layer	Each module's dependencies.py builds the Repository -> Service chain via Depends(), exposed as named Annotated[...] aliases (e.g. TicketServiceDependency).
Service Layer	Business rules, validation, uniqueness checks, state-machine transitions, and raising typed domain exceptions. Depends only on repositories.
Repository Layer	SQLAlchemy queries and persistence only — no validation, no exception translation beyond what the ORM itself raises.
Database Layer	PostgreSQL in production (SQLite in the test suite via dependency override), accessed exclusively through the shared SQLAlchemy engine/session in app/database/.
Infrastructure Layer	app/core, app/common, app/config — settings, logging, the exception-handler registry, and shared dependency aliases that every other layer builds on.
AI Layer	app/ai/* — a parallel set of vertical slices following the identical Router -> Service -> Repository -> Database pattern, plus the LLM provider abstraction described in Section 4.
Background Workers	app/workers/ — reserved for Celery task definitions; currently scaffolding only (Section 19).

Figure 6.1 — Layered Request Flow


PRESENTATION LAYER - app/<module>/router.py
FastAPI APIRouter - path, method, status code, HTTP translation
Depends(...)
DEPENDENCY LAYER - app/<module>/dependencies.py
Builds Repository -> Service chain from the request-scoped
database session (app.core.dependencies.DatabaseDependency)
SERVICE LAYER - app/<module>/service.py
Business rules, validation, domain exceptions
REPOSITORY LAYER - app/<module>/repository.py
SQLAlchemy queries only, no business rules
DATABASE LAYER - app/database/ (engine, session)
PostgreSQL (prod) / SQLite (test, via dependency override)
Cutting across every layer above:
INFRASTRUCTURE app/core app/common app/config
AI LAYER app/ai/* (same 4-layer shape, parallel stack)
BACKGROUND WORK app/workers (scaffolding only)

7. Request Lifecycle

This section traces a single authenticated, permission-guarded request from the wire to the database and back, using `POST /api/v1/tickets` as the concrete example (one of the four routers that actually uses the full RBAC guard chain).

7.1 Step-by-Step Trace

Step	What Happens
1. HTTP Request	Client sends POST /api/v1/tickets with a JSON body and an Authorization: Bearer <token> header.
2. Router	app/api/v1/router.py has already mounted ticket_router at collection time; FastAPI's routing matches path + method to app/tickets/router.py's create_ticket handler.
3. Authentication	The handler's TicketCreatePermission parameter triggers Depends(require_permission("ticket", "create")), which itself depends on CurrentActiveUserDependency -> get_current_user -> decodes the bearer token via app.auth.jwt.decode_access_token, loads the User by the token's sub claim.
4. Authorization	require_permission's inner dependency checks current_user.is_superuser (bypass) or calls RBACService.has_permission(user_id, "ticket", "create"); raises HTTP 403 if neither holds.
5. Dependencies	TicketServiceDependency resolves: DatabaseDependency (a fresh SQLAlchemy Session from get_db) -> TicketRepository(session) -> TicketService(repository).
6. Service	TicketService.create_ticket(TicketCreate, organization_id, created_by) validates title uniqueness via the repository, constructs a Ticket domain object with is_active=True.
7. Repository	TicketRepository issues the SQLAlchemy INSERT and returns the persisted Ticket ORM object — no business logic here.
8. SQLAlchemy / Database	The session flushes the INSERT to PostgreSQL (or SQLite in tests) through the shared engine; the transaction is committed by the repository/service layer.
9. Response	The router maps the returned Ticket to a TicketResponse schema and FastAPI serializes it to JSON with HTTP 201; on any raised AppException subclass, the global handler (Section 14) converts it to a structured ErrorResponse instead.

Figure 7.1 — Request Lifecycle Sequence

Figure 7.1 — The authenticated, permission-guarded path. Most routers (17 of 21) omit the RBAC step and depend only on CurrentActiveUserDependency, or on no auth dependency at all — see Section 5 and Section 8.

8. Authentication Flow

8.1 JWT Authentication

Authentication is stateless bearer-token JWT. `POST /api/v1/auth/register` creates a user (email/username uniqueness enforced, password hashed before storage); `POST /api/v1/auth/login` verifies credentials and returns `TokenResponse{access_token, token_type: "bearer"}`. Every protected endpoint declares an `OAuth2PasswordBearer(tokenUrl="/auth/login")` scheme and extracts the bearer token automatically.

> **■ Verified Finding — Two Parallel, Inconsistent JWT Implementations**
>
> Two independent token modules exist in this codebase. `app/config/security.py` reads its secret, algorithm, and expiry from `Settings` (i.e. from `.env`/`JWT_SECRET_KEY`) — but nothing in the live authentication path calls it; it is dead code from the login/verification flow's perspective. `app/auth/jwt.py` is the module actually imported by `app/auth/service.py` and `app/auth/dependencies.py`, and it hardcodes its own module-level constants — `SECRET_KEY = "your-secret-key-change-in-production"`, `ALGORITHM = "HS256"`, `ACCESS_TOKEN_EXPIRE_MINUTES = 30` — completely independent of `Settings`. Setting `JWT_SECRET_KEY` in `.env` currently has **no effect** on tokens actually issued or verified. This must be reconciled before production deployment — see Section 19.

8.2 Current User & Current Active User

`get_current_user` (app/auth/dependencies.py) decodes the bearer token via `app.auth.jwt.decode_access_token`, catches any exception as a generic `401 "Invalid authentication credentials."`, parses the token's `sub` claim as a user UUID, and loads the user via `UserRepository.get_by_id()` (raising `401 "User not found."` if missing).

> **■ Verified Finding — 'Active User' Check Is Not Actually Enforced**
>
> There is no separate `get_current_active_user` function. `CurrentActiveUserDependency` is simply an alias for `Annotated[User, Depends(get_current_user)]` — `get_current_user` never checks `user.is_active`. A deactivated user holding a still-valid (unexpired) token can continue to authenticate successfully. `get_current_superuser` does build on this dependency to add a genuine `is_superuser` check, raising `403` if false.

8.3 Password Hashing

Both `app/auth/password.py` and `app/config/security.py` independently instantiate `pwdlib.PasswordHash.recommended()` (the same library, duplicated rather than shared) and expose `hash_password`/`verify_password`. Note this contradicts the project's declared dependency manifest: `pyproject.toml` lists `passlib[bcrypt]` and `python-jose[cryptography]`, but no source file imports either — the code actually runs on `pwdlib` (installed, undeclared) and `PyJWT` (installed, undeclared) instead. This manifest drift should be reconciled — see Section 19.

8.4 Access Tokens & Refresh Tokens

Access tokens are issued with a 30-minute expiry (`sub` + `exp` claims only, via `app/auth/jwt.py`). **There is no refresh-token mechanism** — once an access token expires, the client must call `/auth/login` again with credentials. `POST /auth/logout` is a stateless no-op (`return None`, 204) — there is no server-side token blacklist or revocation store, so a logged-out token remains valid until its natural expiry.

8.5 Permissions & RBAC

See Section 5 (RBAC module) and Section 7 for the full mechanism. In summary: `require_permission(resource, action)` is a dependency factory used as a router guard, checked against the four-table Permission/Role/RolePermission/UserRole model, with superusers bypassing the check entirely. Adoption is partial: only 4 of the 21 mounted routers (tickets, notifications, users) use it — the remainder rely on a bare active-user check or, in several cases (projects, audit, analytics, sla, workflows, most of files/email), no authentication dependency at all.

8.6 Organization Isolation

> **■ Verified Finding — Multi-Tenancy Is Not Automatically Enforced**
>
> `get_current_user` does not filter or scope anything by `organization_id`. Every business query that must stay within a tenant boundary does so only because that module's repository/service explicitly filters by `current_user.organization_id` — there is no dependency, middleware, or base-repository mechanism that injects this filter automatically. RBAC`Service.assign_role` is a concrete example of a module that does check organization match explicitly (`user.organization_id != role.organization_id` raises a conflict); other modules must be individually audited for the same discipline. This is a systemic risk worth a dedicated security review before handling production tenant data.

8.7 Security Best Practices — Current State vs. Recommended

Concern	Current State	Recommendation
JWT secret source	Hardcoded in app/auth/jwt.py	Load from Settings/.env; remove the unused app/config/security.py duplicate or make it the single source of truth.
Active-user enforcement	Not checked by get_current_user	Add an explicit is_active check before returning the user.
Token revocation	None — logout is a client-side no-op	Introduce a token blacklist/short-lived-token + refresh-token pattern for real logout semantics.
Organization scoping	Manual, per-repository	Consider a base-repository or dependency-level enforcement so tenant isolation cannot be forgotten in a new module.
RBAC adoption	4 of 21 routers	Extend require_permission coverage to every mutating endpoint, especially projects/audit/analytics/sla/workflows which currently have no guard at all.
Dependency manifest	passlib/python-jose declared, unused; pwdlib/PyJWT used, undeclared	Reconcile pyproject.toml dependencies with actual imports.

9. Database Architecture

9.1 Models

27 tables are live in the current schema (reachable via the application's import graph), plus 2 additional tables (`ai_conversations`, `ai_conversation_messages`) defined by the `chat` module that only materialize if something imports `app.ai.chat.models` — which nothing in the live application currently does, since the chat router is unmounted (Section 4). Every table inherits from a shared `DeclarativeBase` (`app/database/base.py`) with a single, Alembic-safe naming convention applied to every index, unique constraint, check constraint, and foreign key.

9.2 UUID Strategy

Every primary key is a UUID4, generated application-side at insert time. The custom `GUID` column type (`app/database/types.py`) makes this portable across dialects: it uses PostgreSQL's native `UUID` type in production and falls back to a `CHAR(36)` string representation on SQLite, which is what allows the entire test suite to run against a local SQLite file with zero model changes.

9.3 Timestamps & Soft Deletes

`TimestampMixin` provides `created_at`/`updated_at` with server-side defaults (`func.now()`, with `onupdate`). Soft-delete is implemented as an `is_deleted` boolean plus a nullable `deleted_at` timestamp. Note the mechanism is **duplicated, not shared**: the five centrally-modeled entities (Organization, User, Team, Project, Ticket) plus Workflow inherit a shared `BaseModel(TimestampMixin, Base)` that provides `id`/`is_deleted`/`deleted_at`/`soft_delete()`/`restore()` once; the eleven module-owned entities (Customer, Comment, Attachment, Notification, AuditLog, File, Email, SLAPolicy, SLAEvent, and the AI tables) instead hand-roll their own `id`/`is_deleted`/`soft_delete()` directly on `TimestampMixin + Base`. Field naming stays consistent, but the implementation is copy-pasted rather than inherited.

9.4 Foreign Keys & Cascade Behavior

`OrganizationMixin` supplies a standard `organization_id` FK with `ondelete="RESTRICT"`, used by User, Project, and Role. Most other modules (Ticket, Team, Customer, Comment, Attachment, Notification, AuditLog, SLAPolicy, File, Email, Workflow) declare their own `organization_id` column by hand, typically with `ondelete="CASCADE"` instead — an inconsistency in cascade semantics between the centrally-modeled 'core' tables and the business-module-owned tables that is worth standardizing in a future revision.

9.5 Indexes

- Uniqueness indexes: Organization.name/code, User.email/username, Ticket.title (global), Team.name (per-organization)/code, Customer.email, Project.name/key, File.storage_key/checksum, Attachment.storage_key/checksum.
- Composite unique constraints (join tables): RolePermission(role_id, permission_id), UserRole(user_id, role_id), Permission(resource, action).
- Lookup indexes: AuditLog.action/entity_type/request_id, Workflow.name/trigger/is_active, Email.status, File.checksum/status.

9.6 Transactions

`app/database/session.py`'s `get_db()` yields a session and closes it in a `finally` block — it does **not** auto-commit or auto-rollback around the request. Commit/rollback discipline is the caller's (repository or service layer's) responsibility, applied consistently per-module (e.g. repositories typically call `session.commit()` after a write and `session.refresh()` before returning).

9.7 Repository Pattern — Applied to Data Access

See Section 12 for the pattern in general. At the database level specifically: a generic `BaseRepository[ModelType]` (`app/repositories/base.py`) provides common CRUD, and several shared-entity repositories build on it directly.

> ■ **Verified Finding — Duplicate OrganizationRepository, and No Committed Migrations**
>
> Two independent, live `OrganizationRepository` classes exist: `app.organizations.repository.OrganizationRepository` (hand-written CRUD, used by `organizations` and `users`) and `app.repositories.organization.OrganizationRepository(BaseRepository[Organization])` (used by `teams`). Both class docstrings explicitly acknowledge the other and state that consolidating them requires a method-by-method behavioral diff that has not been performed — for example, one filters soft-deleted rows via `deleted_at.is_(None)`, the other via `is_deleted.is_(False)`. They were left deliberately separate rather than risk a silent behavior change. Separately: the Alembic environment is fully configured (`app/database/migrations/env.py`) but no `versions/` directory or `alembic.ini` exists in the repository — the schema in every environment today, including tests, is produced by `Base.metadata.create_all()`, not by a committed migration history. A production deployment must generate an initial migration before this platform can be safely upgraded in place.

10. AI Request Flow

This section restates the pipeline from Section 4 as a request-flow diagram, explicit about which arrows represent real, wired calls today versus intended-but-not-yet-connected steps.

Figure 10.1 — Document-to-Answer Workflow (Intended vs. Actual)

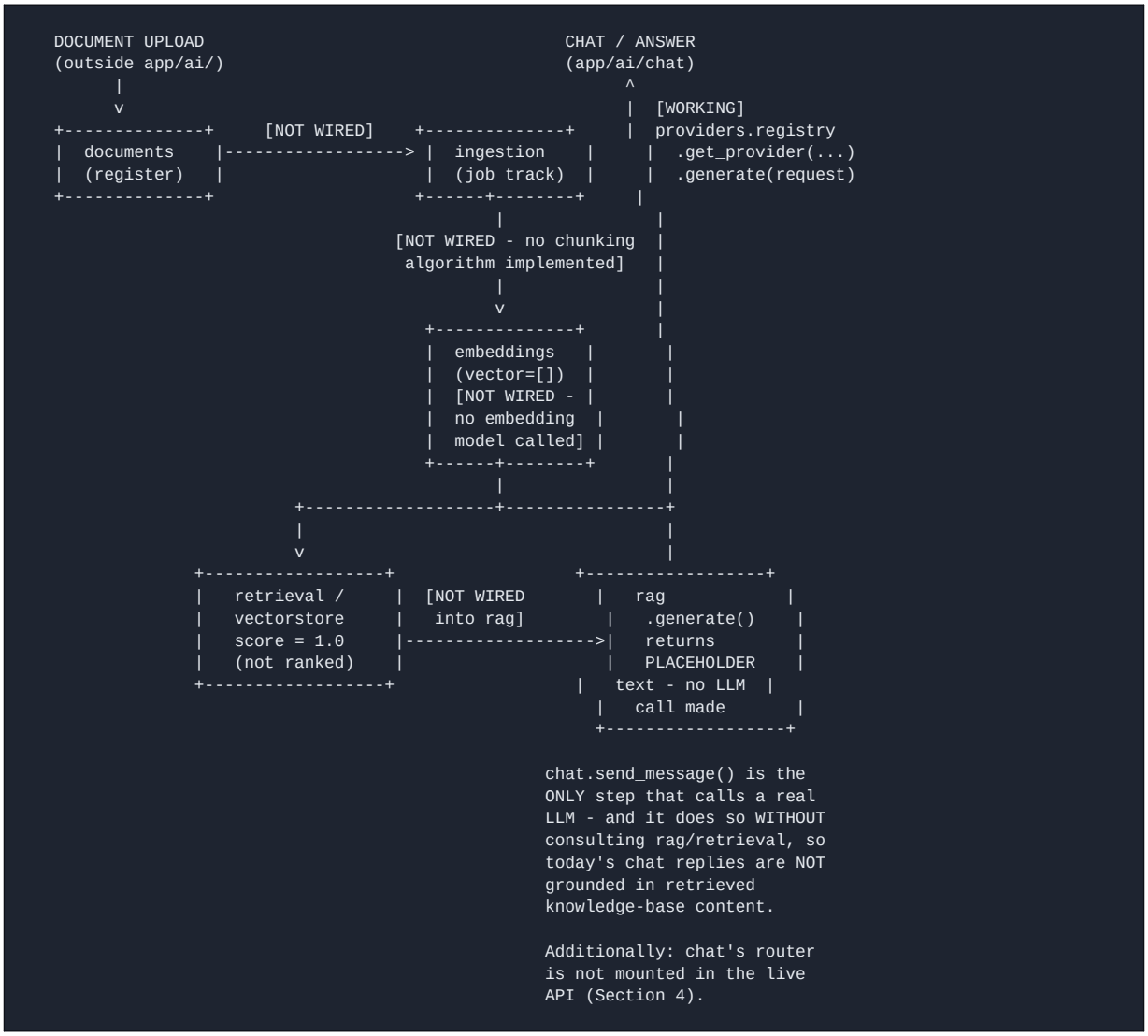


Figure 10.1 — Solid business intent (typed models, endpoints) exists for the full pipeline. The arrows marked [NOT WIRED] are the gap between today's implementation and a functioning RAG pipeline.

10.1 Step Detail

Pipeline Step	Current Implementation
Document Upload	Handled outside app/ai/ (by files/attachments); only a storage_path and checksum are expected as input to registration.
Ingestion	IngestionService tracks a job's status and counters; no code reads a document's actual content.

Pipeline Step	Current Implementation
Chunking	Not implemented anywhere in app/ai/. chunk_size/chunk_overlap are stored configuration values, not applied by any algorithm.
Embedding Generation	AIEmbeddingService.create() persists a row with an empty vector and PENDING status; no embedding-model provider call is made.
Vector Storage	Is the embeddings table itself — there is no separate vector-index write step.
Retrieval	retrieval and vectorstore both page through embedding rows with a hardcoded score of 1.0 — not a ranked similarity search.
RAG	rag.generate() builds citations from whatever embeddings it fetched directly (bypassing retrieval/vectorstore) and returns a hardcoded placeholder answer string; no LLM provider is called.
LLM / Response Generation	Only chat.ConversationService.send_message() makes a real LLM call, via app.ai.providers.registry — but grounded only in conversation history, not in rag/retrieval output, and its router is not mounted on the live API.

11. Dependency Injection

Every module wires its Repository and Service through FastAPI's native `Depends()` system, in a `dependencies.py` file with a consistent shape across all 27 modules (17 business + 10 AI).

11.1 Database Session

The root of every dependency chain is `app.core.dependencies.DatabaseDependency` (an alias for `Annotated[Session, Depends(get_db)]`), itself wrapping `app.database.session.get_db()` — a generator that opens one `Session` per request and closes it when the request completes, regardless of success or failure.

11.2 Repository Dependencies

Each module defines a getter, e.g. `get_ticket_repository(db: DatabaseDependency) -> TicketRepository`, that simply constructs the repository with the injected session. This is the only place a repository is ever instantiated — routers and services never construct one directly.

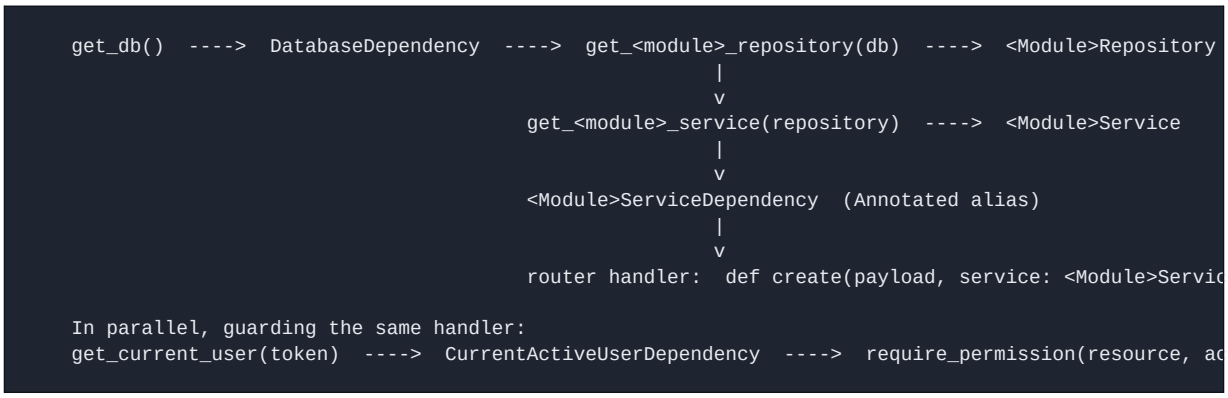
11.3 Service Dependencies

Each module then defines `get_ticket_service(repository: Annotated[TicketRepository, Depends(get_ticket_repository)]) -> TicketService`, and exposes the whole chain to routers as a single named alias, e.g. `TicketServiceDependency = Annotated[TicketService, Depends(get_ticket_service)]`. A router handler then only ever needs one parameter — the service alias — to receive a fully-constructed, request-scoped service instance.

11.4 Authentication Dependencies

`CurrentActiveUserDependency`, `CurrentSuperuserDependency`, and `require_permission(resource, action)` (Section 8) compose in exactly the same way — they are themselves `Depends()`-based, so they can be layered into any router's parameter list alongside a service dependency without any special-casing.

Figure 11.1 — Dependency Composition Chain



11.5 Advantages of This Approach

- Every layer is independently testable — a service can be instantiated with a mocked repository with zero FastAPI machinery involved (see Section 15).
- No service or repository is ever a global singleton holding a stale database session — a fresh session is created and torn down per request.
- Cross-cutting concerns (auth, permissions, pagination) compose by simple parameter addition, not by modifying a shared base controller class.
- The dependency graph for any endpoint is fully visible in its function signature — there is no hidden service-locator lookup to trace.

12. Repository Pattern

Every module's `repository.py` is the single place SQLAlchemy is imported and queried for that module's entities. A generic `BaseRepository[ModelType]` (`app/repositories/base.py`) exists and is used directly by some shared-entity repositories (e.g. the `app.repositories.organization` variant, `app.repositories.project`, `app.repositories.user`); most business-module repositories instead hand-write their own CRUD methods against the same conventions without formally subclassing it.

12.1 CRUD

Every repository exposes the same shape: `create`, `get` (by ID), `list` (paginated), `update`, and `delete` (soft, where the model supports it). Domain-specific lookups (e.g. `get_by_email`, `exists_by_checksum`, `get_by_slug`) are added per-repository as needed.

12.2 Transactions

Repositories are responsible for calling `session.commit()` after a mutating operation and `session.refresh()` to reload server-generated values (e.g. `created_at`) before returning the ORM object to the service layer.

12.3 Pagination

The shared `app.common.pagination.PaginationParams` dependency (`page`, `page_size`, computed `offset`/`limit`) is accepted by most list endpoints and translated into a SQLAlchemy `.offset().limit()` call at the repository layer; results are wrapped in the shared `PaginatedResponse[T]` envelope (`app.common.responses`) with a computed `total_pages`.

12.4 Filtering & Search

Simple filters (status, organization_id, active_only) are implemented as optional keyword arguments on `list()` methods, translated to SQLAlchemy `.where()` clauses. A handful of modules (audit, files, email) expose a dedicated `/search` endpoint backed by a repository method that builds a dynamic filter set from a structured search schema.

12.5 Statistics

Several modules (documents, ingestion, vectorstore, retrieval, rag, analytics) expose a `/statistics`-style endpoint backed by `func.count()` aggregate queries at the repository layer, rather than loading and counting full result sets in Python.

12.6 Benefits

- Business services never see a SQLAlchemy Query object or session directly — they only see typed method calls and ORM/domain objects.
- Swapping the underlying persistence technology for a single module would only require rewriting that module's repository.py.
- Repository-level unit tests can assert on exact query behavior (uniqueness checks, soft-delete filtering) independent of business rules.

13. Service Layer

Every module's `service.py` is where business rules live. Services depend only on repositories (their own, and occasionally another module's, as in RBAC's checks against User/Role) and on the shared exception hierarchy — never on FastAPI request/response objects, and never on another module's router.

13.1 Business Rules

Examples verified across modules: uniqueness enforcement (Organization.name/code, User.email/username, Ticket.title, Customer.email, Team.name/code), state-machine transitions (Email.send/retry/cancel, SLAEvent.record_first_response/resolve, Workflow.activate/deactivate), and cross-entity validation (Team creation validates its parent Organization and optional lead User exist; RBAC's assign_role validates the user and role share an organization).

13.2 Validation

Structural validation (types, required fields, string length/format) is handled by Pydantic at the schema layer before a service method is ever called. Services layer on *semantic* validation that Pydantic cannot express — uniqueness against existing rows, referential existence of a related entity, and business state preconditions (e.g. refusing to assign an inactive SLA policy, or to cancel an already-sent email).

13.3 Exception Handling

Services raise typed exceptions rather than returning error codes or `None` on failure. Most modules define their own exception subclasses (see Section 14) that either extend the shared `app.core.exceptions.AppException` hierarchy directly, or, in a few modules (projects, teams' own errors), define a fully independent module-local exception set with its own HTTP-status mapping performed in the router rather than the global handler.

13.4 Response Models

Services return ORM model instances or module-internal domain objects — never a pre-built HTTP response. The router layer is exclusively responsible for mapping a service's return value into the module's Pydantic response schema, which keeps the same service method reusable regardless of how its result will eventually be serialized.

13.5 Separation of Concerns

The practical test applied throughout this codebase: a service method should be fully exercisable in a unit test with a mocked repository and zero FastAPI, HTTP, or database machinery in scope. Section 15 confirms this is exactly how the test suite is structured — service-layer tests inject a `MagicMock(spec=<Repository>)` and assert purely on business-rule outcomes.

14. Exception Architecture

14.1 Application Exceptions — the Shared Base Hierarchy

app.core.exceptions.AppException is the root of the shared exception tree. Every subclass carries a message and an HTTP status code, so the global handler can translate any of them into a structured response without per-exception-type handler code.

Exception	HTTP Status	Usage
AppException	500	Base class. Stores .message and .status_code.
ValidationException	422	Default message: "Validation failed."
AuthenticationException	401	Failed or missing credentials.
AuthorizationException	403	Default message: "Access denied."
ResourceNotFoundException	404	Takes a resource name; message = "{resource} not found."
ConflictException	409	Uniqueness violations, illegal state transitions.
DatabaseException	500	Wraps unexpected persistence failures.
AIServiceException	503	Default message: "AI service unavailable."

14.2 Module Exceptions

Most modules define their own exception subclasses in a local exceptions.py, either subclassing the shared AppException hierarchy directly (e.g. comments.CommentError(AppException)) or, in a smaller number of modules (notably projects and the AI submodules), defining a fully independent exception set whose HTTP status mapping is performed locally in the router's except clauses rather than delegated to the global handler. Both patterns coexist in the codebase; the shared-hierarchy pattern is the majority and the recommended one for new modules, since it lets the global handler do the HTTP translation uniformly.

14.3 HTTP Status Mapping

For modules using the shared hierarchy, status mapping is automatic — the exception's own .status_code is used directly by the global handler. For modules with independent exception sets, the router explicitly catches each exception type and re-raises as the appropriate fastapi.HTTPException with a hardcoded status code, which duplicates the status-code decision in every such router.

14.4 Global Exception Handler

app.common.exceptions.register_exception_handlers(app) is called once in app/main.py and installs three handlers via app.add_exception_handler():

- **AppException** → logs a warning, returns a JSON ErrorResponse(message, error_code=exception class name) with the exception's own status_code.
- **RequestValidationError** (Pydantic/FastAPI validation failures) → HTTP 422, error_code="REQUEST_VALIDATION_ERROR", with the raw Pydantic error list attached under details.errors.

- **Exception** (catch-all) → logs a full traceback via logger.exception(), returns a generic HTTP 500 ErrorResponse with error_code="INTERNAL_SERVER_ERROR" — this is the last line of defense that ensures no unhandled exception ever leaks a raw traceback to a client.

15. Testing Architecture

622+ test functions across 90+ test files, mirroring the source tree one folder per module, executed by Pytest with `testpaths = ["app/tests"]`.

15.1 Pytest & Fixtures

`app/tests/conftest.py` (~1070 lines) provides the shared fixture graph. Its central fixtures:

Fixture	What It Does
setup_database (autouse)	Recreates the full schema (create_database()/drop_database()) around every single test — full isolation at the cost of per-test setup time.
db_session	A session from the SQLite test database, rolled back and closed after each test.
client	A TestClient with app.dependency_overrides[get_db] = get_db_session applied before construction — this is the mechanism that redirects router tests to the SQLite test database instead of the real Postgres one.
auth_headers	Performs a genuine login round-trip (POST /api/v1/auth/login with real seeded credentials) through the client fixture and returns a real bearer-token header — not a synthetically minted token.
authenticated_client	Convenience fixture combining client + auth_headers into one pre-authenticated client.
Per-domain factories	organization, user, ticket, email/email_factory, file/file_factory, sla_policy/sla_policy_factory, workflow, plus AI-domain fixtures for conversations, embeddings, documents, knowledge bases, and ingestion jobs.

15.2 The Test Database

`app/tests/database.py` defines a completely separate SQLite database (`TEST_DATABASE_URL = "sqlite:///./test.db"`, file-based, not in-memory) with its own engine and session factory, entirely independent of the production `DATABASE_URL` setting. This is what allows the full test suite to run in any environment without a live PostgreSQL server — the custom `GUID` type (Section 9) transparently falls back to string storage on SQLite so no model code needs to know which database it's talking to.

15.3 Repository Tests

Query the real (SQLite) database directly through fixtures — no mocking. These tests assert on actually persisted rows and verify soft-delete filtering, uniqueness constraint behavior, and query correctness at the SQL level.

15.4 Service Tests

Inject a `MagicMock(spec=<Repository>)` into the service under test and assert on business-rule outcomes (raised exceptions, correct repository method calls) without touching a database at all.

15.5 Router Tests

Use `fastapi.testclient.TestClient` against the real `app.main.app` instance, most commonly via the shared `client` fixture (which redirects `get_db` to SQLite), though some modules construct their own module-level `TestClient(app)` and override only their service dependency — a pattern that, if the endpoint's auth path also needs a database lookup, can leave that path pointed at the real (and in a sandboxed environment,

unreachable) production database rather than the test one. This distinction matters operationally and is documented as a known verification pitfall in Section 21.

15.6 Authentication Tests

app/tests/auth/ (5 files) covers registration, login, token creation/decoding, and password hashing directly, including dedicated test_jwt.py and test_password.py files beyond the standard three-file pattern.

15.7 Integration Tests & Dependency Overrides

Router-level tests are the project's integration tests — they exercise the full Router → Service → Repository → Database chain for real, with FastAPI's `app.dependency_overrides` dict used surgically to substitute only the database session (and sometimes a mocked service) while leaving the rest of the dependency chain — including authentication and permission checks — genuinely exercised.

15.8 Testing Strategy Summary

Layer	Database	Mocked	What It Verifies
Repository	Real DB (SQLite)	None	Query correctness, constraints, soft-delete filtering
Service	None	Repository (MagicMock)	Business rules, validation, exception raising
Router	Real DB (via override)	Sometimes the service	End-to-end HTTP behavior, status codes, auth/permissions

16. Code Quality

16.1 Ruff

Configured in the repo-root `pyproject.toml` with rule families `E, F, I, UP, B, C4, SIM, N, ANN, D` (pycodestyle errors, Pyflakes, import sorting, pyupgrade, bugbear, comprehensions, simplify, naming, annotations, docstrings), with `D203, D212, B008, N818, N107` deliberately ignored and Google-convention docstrings enforced. Test files get a relaxed docstring/annotation profile via `[tool.ruff.lint.per-file-ignores]`. The full codebase passes with zero issues as of this revision.

16.2 Black

88-column line length, Python 3.14 target. The full codebase is fully formatted with zero pending changes as of this revision.

16.3 MyPy

Runs in `strict` mode: `disallow_untyped_defs`, `disallow_incomplete_defs`, `check_untyped_defs`, `warn_return_any`, `warn_unused_ignores`, `no_implicit_optional` all enabled. The full codebase passes with zero issues across 427 source files as of this revision — every function signature in the application is fully typed.

16.4 Typing Standards

Modern syntax throughout: PEP 604 unions (`str | None` rather than `Optional[str]`), PEP 695 generic classes (`class SuccessResponse[T](BaseModel)`), and `from __future__ import annotations` in every module to keep annotations lazily evaluated.

16.5 Google Docstrings

Every non-trivial public function/class documents its purpose, arguments, and return value in Google docstring convention, enforced by Ruff's `pydocstyle` integration.

16.6 SOLID & Clean Code

See Section 2 for how each SOLID principle maps onto concrete structures in this codebase (provider abstractions for Open/Closed and Liskov Substitution, per-module dependency injection for Dependency Inversion, and so on). In practice, the enforcement mechanism for clean code here is structural rather than aspirational: the one-module-per-folder, one-file-per-responsibility shape makes most SOLID violations visually obvious (a service importing a router, a repository containing business rules) even without a dedicated architecture linter.

17. Project Statistics

Metrics below were measured directly against the repository at this revision.

Metric	Value
Python source files (app/, excl. tests)	337
Python test files	90
Total Python files in app/	427
Lines of code — application (excl. tests)	~23,747
Lines of code — tests	~14,782
Business modules	17
AI submodules	10
Routers mounted in the live API	21
Unique API paths	87
Total API operations (path + method)	136
Live database tables	27 (+2 dormant chat tables — see Section 9)
Test functions	622+
Ruff issues (this revision)	0
Black formatting changes needed (this revision)	0
MyPy errors, strict mode (this revision)	0

17.1 Architecture Patterns in Use

- Clean Architecture (vertical feature slices over horizontal technical layers)
- Repository Pattern (data access isolated per module)
- Service Layer (business rules isolated per module)
- Dependency Injection (FastAPI Depends() throughout)
- Provider / Strategy Pattern (AI providers, storage providers, email providers — all swappable behind an abstract base)
- Global Exception Handling (three-tier handler registered once, Section 14)
- Soft Delete (is_deleted / deleted_at rather than hard deletes, throughout)
- Multi-Tenancy via Organization scoping (manually enforced per-module — Section 8)

18. Complete End-to-End Flow

This section walks one realistic user journey through the platform, citing the actual endpoints and services involved at each step, and noting explicitly where the journey runs into the implementation gaps documented in Section 4.

Figure 18.1 — End-to-End Journey Sequence

Figure 18.1 — Every request shown is a real, working endpoint except the final chat step, which is implemented but not mounted, and which — even if mounted — does not consult the retrieval result obtained earlier in this same journey.

18.1 Narrative Walkthrough

- **Login** — POST /api/v1/auth/login authenticates and returns a bearer token (Section 8).
- **Create Organization** — a superuser calls POST /api/v1/organizations to establish the tenant boundary every subsequent record will be scoped to.
- **Create Project** — POST /api/v1/projects associates a project with the organization.
- **Upload Document** — the document's bytes are stored outside app/ai/ (via files/attachments); POST /api/v1/ai/documents registers its metadata.
- **AI Ingestion** — POST /api/v1/ai/ingestion creates a job to track processing; as documented in Section 4, no chunking algorithm actually runs against the document's content at this step today.
- **Embedding Creation** — POST /api/v1/embeddings persists an embedding record; as documented in Section 4, no embedding-model provider is called, so the stored vector is empty.

- **Vector Storage** — is implicit: the embeddings table itself is the vector store.
- **Knowledge Retrieval** — POST /api/v1/ai/retrieval/retrieve returns results, but with a hardcoded relevance score rather than genuine semantic ranking.
- **Ticket Creation** — POST /api/v1/tickets creates the support case this whole journey is in service of.
- **AI Assistant** — the one step that produces a real LLM-generated reply is app.ai.chat.ConversationService.send_message, called via a router that is not currently mounted on the live application.
- **Response Generation** — if chat were mounted and invoked, the response would come from a genuine provider call (OpenAI or the Mock provider), but grounded only in prior conversation turns and a static system prompt — not in the retrieval result obtained a few steps earlier in this same journey.

19. Deployment Architecture

19.1 FastAPI

The application is a standard ASGI app (`app.main.app`) intended to run under `uvicorn[standard]` (a declared dependency). `Settings.HOST` (default `0.0.0.0`) and `Settings.PORT` (default `8000`) configure the bind address; `app.core.lifespan.startup/shutdown` run through the app's `lifespan` context manager.

19.2 Docker

> **■ Verified Finding — docker-compose.yml References a Dockerfile That Does Not Exist**
>
> The repo-root `docker-compose.yml` defines three services — `backend` (`build: {context: ., dockerfile: Dockerfile}`, port 8000, depends on postgres and redis), `postgres` (image `postgres:17`, port 5432), and `redis` (image `redis:8`, port 6379). No `Dockerfile` exists anywhere in the repository (checked both the repo root and `backend/`) — running `docker compose up --build` would fail on the `backend` service as the compose file stands today. Writing this Dockerfile is a prerequisite for containerized deployment and is the first item on the roadmap in Section 20.

19.3 PostgreSQL

Production datastore, provisioned by `docker-compose.yml`'s `postgres:17` service (credentials `support`/`support`, database `support_db`, matching `Settings.DATABASE_URL`'s default). As documented in Section 9, no migration history is committed yet — provisioning currently relies on `Base.metadata.create_all()`, which is acceptable for development but not for a production upgrade path.

19.4 Workers

`app/workers/` contains 5 files intended to define Celery tasks (analytics, document, email, ticket task modules, plus `celery_app.py`) backed by the `redis` service as broker. As verified in Section 4 and restated here: every one of these files is currently a 3-line stub (a docstring and a future-annotations import) with no `Celery(...)` application instance and no `@app.task` definitions. Background/async processing is architecturally reserved but not implemented.

19.5 Environment Variables

`Settings` (Pydantic `BaseSettings`) reads from `.env` at the repo root. Key variables, all documented in `.env.example`: `APP_NAME`/`APP_ENV`/`DEBUG`, `DATABASE_URL`, `REDIS_URL`, `JWT_SECRET_KEY`/`JWT_ALGORITHM`/`ACCESS_TOKEN_EXPIRE_MINUTES` (noting the Section 8 caveat that this JWT_SECRET_KEY has no effect on the live auth path today), `LOG_LEVEL`, and a full block of AI-provider credentials (OpenAI, Anthropic, Gemini, Groq, Ollama, Azure OpenAI).

19.6 Production Deployment Checklist

Before this platform is deployed to production, the following gaps identified in this document should be closed:

- Write the missing Dockerfile referenced by docker-compose.yml.
- Generate and commit an initial Alembic migration; stop relying on Base.metadata.create_all() outside tests.

- Reconcile the two parallel JWT implementations into one, sourced from Settings/.env (Section 8).
- Enforce is_active in the authentication dependency chain.
- Reconcile the dependency manifest (pyproject.toml) with actual runtime imports (pwdlib/PyJWT vs. declared passlib/python-jose).
- Decide whether to complete or remove the AI pipeline's unimplemented steps (chunking, embedding generation, real vector similarity, RAG-grounded chat) before advertising them as functional — see Section 20.
- Mount app.ai.chat's router if the conversational feature is intended to be customer-facing.

20. Future Roadmap

This roadmap is ordered by dependency — later items generally build on earlier ones being in place.

Initiative	Scope
AI Pipeline Completion	Implement real document chunking, call an embedding-model provider in embeddings.create(), replace hardcoded similarity scores in retrieval/vectorstore with real vector comparison, wire rag.generate() to call retrieval + a provider, and connect chat.send_message() to rag/retrieval so replies are actually knowledge-grounded. Mount the chat router. This is the highest-value gap identified in this document — it is the difference between an architected RAG platform and a functioning one.
AI Agents	Multi-step, tool-using agents (e.g. an agent that can look up a ticket, check SLA status, and draft a response) built on top of a completed RAG pipeline.
Realtime Chat	WebSocket or Server-Sent-Events support for live agent/customer conversations, building on the already-implemented streaming provider support (app.ai.providers' /stream endpoint).
Multi-LLM	Complete the provider implementations currently stubbed with NotImplementedError (Anthropic, Gemini, Groq, Ollama, Azure OpenAI) so the multi-vendor abstraction already in place is actually multi-vendor in practice.
Frontend	A web/mobile client consuming this API — none exists in this repository beyond a placeholder frontend/ directory at the repo root.
Background Workers	Implement the Celery task stubs in app/workers/ for asynchronous document processing, scheduled SLA breach detection, and outbound email/notification delivery.
Monitoring	Wire the already-declared prometheus-client dependency to an actual metrics endpoint, and the already-declared structlog dependency to app/config/logging.py (currently plain stdlib logging).
CI/CD	Automate the Ruff/Black/MyPy/Pytest gates already defined in this codebase's tooling configuration into a continuous integration pipeline, plus automated Alembic migration checks once migrations exist.
Production Deployment	Close the Section 19 checklist (Dockerfile, migrations, JWT consolidation) before any production rollout.
Kubernetes / Cloud Deployment	Containerize per the completed Dockerfile, then define Kubernetes manifests or a managed-container deployment (ECS/Cloud Run/AKS) once the single-container Docker path is proven.

21. Lessons Learned

This section documents the reasoning behind the architecture's core decisions, so a future engineer questioning “why is it built this way” has the answer without needing to ask the original team.

21.1 Why Repository Pattern

Isolating SQLAlchemy behind a repository per module means a service's business logic can be unit-tested with a mocked repository and zero database machinery, which is exactly the pattern this codebase's test suite uses throughout (Section 15). It also means a query bug is always localized to exactly one file per module, never spread across service methods.

21.2 Why Service Layer

Putting business rules in a layer that knows nothing about HTTP means the same validation logic protects every caller of that logic, not just the one currently wired to a router — relevant given that several service methods in this codebase (e.g. TicketService.update_ticket/delete_ticket, SLAService.assign_policy) are already fully implemented and tested but not yet exposed via a route; when they are exposed, no new validation code will be needed.

21.3 Why Dependency Injection

FastAPI's native Depends() system was chosen over a third-party DI container because it requires no additional framework concept — the dependency graph is just Python function composition, visible directly in each handler's signature, and it integrates natively with FastAPI's own request lifecycle and automatic OpenAPI documentation generation.

21.4 Why AI Separated into app/ai

AI capability was given its own top-level namespace, structured identically to a business module internally (the same Router → Service → Repository → Database shape) but grouped together because AI submodules share vocabulary (provider/model enums, a common request/response schema shape) that no business module needs, and because the AI capability set is expected to grow and change independently of core business CRUD — new providers, new retrieval strategies — without that churn touching ticket or organization code.

21.5 Why Business Modules Remain Independent

Each business module owning its own models/schemas/repository/service/router/dependencies/exceptions/constants means the cost of understanding, testing, or modifying one capability does not scale with the size of the whole platform. The five centrally-shared entities (Organization, User, Team, Project, Ticket) are the deliberate exception to this rule — they exist in app/models/ specifically because more than one module needs to own queries against them, and duplicating them per-module was judged worse than the coupling of a shared location.

21.6 Verified Pitfall — Test Isolation via Global Dependency Overrides

During the preparation of this document, a genuine testing pitfall was directly observed: `app.dependency_overrides` is a single mutable dictionary on the shared `app` object. A test module that builds its own bare `TestClient(app)` at import time and overrides only its own service dependency (rather than using

the shared `client` fixture, which also redirects `get_db` to the SQLite test database) can behave correctly in isolation but produce environment-dependent failures when run as part of the full suite, depending on what state an earlier-running test file left in that shared dictionary. New router tests should prefer the shared `client` fixture over constructing a standalone `TestClient`, specifically to avoid this class of flakiness.

21.7 Benefits of the Final Architecture

- A new engineer can be productive on one business module within a day, without reading the AI layer or any other business module first.
- The dependency direction (Router -> Service -> Repository -> Database) has zero verified violations across the entire codebase — this is a real, load-bearing architectural invariant, not an aspiration.
- Every layer is independently and cheaply testable, which is reflected in a 622+ test, 90+ file suite with a consistent per-layer testing convention.
- The AI layer's incomplete pipeline steps are isolated to their own modules — none of the business-module code depends on the AI layer being complete, so business functionality is unaffected by AI roadmap timing.

22. Conclusion

The Enterprise AI Support Platform backend implements a complete, consistently-structured multi-tenant support platform: 17 business modules covering organizations, people, tickets, and their surrounding workflow (SLA, notifications, audit, analytics), secured by JWT authentication and a role-based permission model, and extended by a 10-module AI package architected around a standard Retrieval-Augmented Generation pipeline. Every module in the system, business or AI, follows the identical Router → Service → Repository → Database layering with dependency injection at every seam, verified in this document to hold with zero exceptions across the codebase.

This document has also recorded, deliberately and specifically, the points where the current implementation does not yet match its architectural intent: two parallel JWT implementations, an unenforced active-user check, partial RBAC adoption, a duplicate OrganizationRepository, no committed database migrations, no Dockerfile, a dependency manifest that has drifted from actual imports, and — most significantly for the platform's core value proposition — an AI retrieval and generation pipeline that is fully modeled and exposed via API but not yet functionally connected end-to-end. Recording these honestly is what makes this document trustworthy as a long-term reference rather than aspirational marketing copy; every finding above was verified by reading the actual source code, not inferred from folder names or documentation comments.

Enterprise AI Support Platform Backend
Version 1.0
ARCHITECTURE LOCKED

This revision of the architecture — the folder structure, the layering convention, the module boundaries described in Sections 3 through 13 — is considered frozen. Structural changes to how modules are organized or how layers communicate should be treated as a new architecture revision requiring its own review, not an incidental side effect of a feature change.

22.1 Future Work Should Focus On

- **Frontend** — no client application exists in this repository today; the API is ready to be consumed.
- **Production Deployment** — close the Section 19 checklist (Dockerfile, migrations, JWT consolidation, dependency manifest).
- **AI Agents** — build on a completed RAG pipeline (Section 20) once the pipeline gaps identified in Section 4 and Section 10 are closed.
- **Monitoring** — activate the already-declared Prometheus and structured-logging dependencies.
- **CI/CD** — automate the quality gates that are already fully configured and passing locally.

End of Document — Enterprise AI Support Platform Backend, Architecture & Technical Design Document, Version 1.0 (Architecture Locked)